



# Topic 02 — Tables & Data Types (Page 1 of 4)

Building the foundation of every database!

"Good Data Builds Great Decisions!"

## 1. What is a Table?

- A table is a collection of related data stored in **rows** and **columns**.
- It helps us organize data in a **structured** format.
- Each table represents a **real-world entity** like students, employees, products etc.

| Column<br>↓ |       |     |        |
|-------------|-------|-----|--------|
| ID          | Name  | Age |        |
| 1           | Rahul | 21  | ← Row  |
| 2           | Priya | 22  | ← Cell |
| 3           | Aman  | 20  |        |

A table stores data like a spreadsheet but much more powerful!

## 2. Parts of a Table

- 1 **Table** — Collection of related data.
- 2 **Column** — A vertical set of values (also called an **attribute** or **field**).
- 3 **Row** — A horizontal set of values (also called a **record**).
- 4 **Cell** — A single value at the intersection of a row and column.

### Quick Definitions

- Table** → Stores related data
- Column** → One property/attribute
- Row** → One complete record
- Cell** → One value

## 3. Example: students table

| student_id | student_name | age | email          |
|------------|--------------|-----|----------------|
| 1          | Rahul        | 21  | rahul@mail.com |
| 2          | Priya        | 22  | priya@mail.com |
| 3          | Aman         | 20  | aman@mail.com  |

- Each row represents one student.
- Each column represents a specific property (ID, name, age, email).
- This table stores information about students in a structured format.
- We can easily search, filter, and analyze this data using SQL.
- This is just the structure + sample data. We will learn how to create this table in the next part!

## 4. Real-world Analogy



Think of a table like a file cabinet in an office.

- Each drawer → a **row** (one record)
- Each label → a **column** (attribute)
- Each piece of information → a **cell** (value)

Just like you store employee files in a cabinet, a database stores data in tables.



## 5. Why Tables are Important?

- ✓ Organize data in a structured way
- ✓ Store large amounts of information
- ✓ Make data easy to search and analyze
- ✓ Represent real-world entities
- ✓ Essential for building relational databases
- ✓ Used in almost every industry (Finance, Healthcare, E-commerce, Education, etc.)



A well-designed table is the first step towards solving real business problems!

Choosing the right data type is the first step towards clean, efficient and scalable databases!

Right Data  
Right Insights  
Brighter Future !

## 1. PostgreSQL Data Types

| Data Type  | Purpose / Description                        | Example             |
|------------|----------------------------------------------|---------------------|
| INTEGER    | Whole numbers (4 bytes)                      | 25                  |
| BIGINT     | Very large whole numbers (8 bytes)           | 9000000000          |
| NUMERIC    | Exact decimal numbers (for money, precision) | 1250.75             |
| TEXT       | Variable length text                         | 'Rahul'             |
| VARCHAR(n) | Variable length text with length limit       | VARCHAR(50)         |
| BOOLEAN    | True or False value                          | TRUE / FALSE        |
| DATE       | Date only (YYYY-MM-DD)                       | 2026-08-26          |
| TIMESTAMP  | Date and time (YYYY-MM-DD HH:MI:SS)          | 2026-08-26 10:30:00 |

## 2. When to Use Which Data Type?

|            |                                                              |
|------------|--------------------------------------------------------------|
| INTEGER    | → For normal whole numbers (e.g., age, quantity)             |
| BIGINT     | → For very large numbers (e.g., user IDs, transaction IDs)   |
| NUMERIC    | → For decimal numbers (e.g., salary, price, amount)          |
| TEXT       | → For general text data (e.g., name, address, description)   |
| VARCHAR(n) | → When you want to set a maximum length (e.g., VARCHAR(100)) |
| BOOLEAN    | → For true/false values (e.g., is_active, is_admin)          |
| DATE       | → For dates only (e.g., date_of_birth, joining_date)         |
| TIMESTAMP  | → For date and time (e.g., created_at, updated_at)           |

Tip: Choose the most appropriate data type based on the kind of data you will store. Don't just use TEXT for everything !

## 3. Practical Examples

| Column Name  | Best Data Type | Reason                  |
|--------------|----------------|-------------------------|
| student_id   | INTEGER        | Whole number, unique ID |
| student_name | TEXT           | Stores text             |
| age          | INTEGER        | Whole number            |
| fees         | NUMERIC        | Decimal value (money)   |
| is_active    | BOOLEAN        | True or False           |
| birth_date   | DATE           | Date only               |
| created_at   | TIMESTAMP      | Date and time           |

## 4. Special Notes in PostgreSQL

- ✓ PostgreSQL supports both VARCHAR(n) and TEXT.
- ✓ For most use cases, TEXT is perfectly fine for storing strings.
- ✓ Use NUMERIC for financial data to avoid rounding errors.
- ✓ BOOLEAN values are TRUE or FALSE (not 1/0 by default).
- ✓ Dates are stored in YYYY-MM-DD format.
- ✓ Timestamps include both date and time.
- ✓ PostgreSQL is case-insensitive for data types (INTEGER and integer are same).

PostgreSQL gives you flexibility! Use it wisely!

## 5. Common Mistakes

- ✗ Storing numbers as TEXT (e.g., salary TEXT)
- ✗ Using INTEGER for decimal values (e.g., price, salary)
- ✗ Forgetting to set an appropriate data type
- ✗ Using unnecessarily small VARCHAR(n)
- ✗ Not considering future data (e.g., too small length)
- ✗ Using TEXT for values that need to be calculated



Wrong data types can cause big problems later !

## 6. Key Takeaways

- ✓ Understand the nature of your data.
- ✓ Choose the most suitable data type.
- ✓ Use NUMERIC for financial/decimal values.
- ✓ TEXT is great for flexible text storage.
- ✓ BOOLEAN, DATE, TIMESTAMP make data more meaningful and usable.
- ✓ Good data types = Better performance + Fewer errors + Scalable database !



"A well-designed table with the right data types makes your database reliable, efficient and future-ready !"



PostgreSQL

# Topic 02 – Tables & Data Types

(Page 3 of 4)

From Design to Implementation

"Structure today,  
Better data tomorrow!"



## 1. Creating a Table

- We use the **CREATE TABLE** statement to create a new table in PostgreSQL.
- It defines the **table** name, column names, and their data types.
- We can also add **constraints** like **PRIMARY KEY**, **NOT NULL**, etc.



### Syntax:

```
CREATE TABLE table_name (
    column1 data_type,
    column2 data_type,
    column3 data_type
);
```

This creates the structure only. It does not insert any data.

## 3. Other Common Constraints

| Constraint         | Purpose                      | Example                                |
|--------------------|------------------------------|----------------------------------------|
| <b>PRIMARY KEY</b> | Uniquely identifies a record | id INTEGER <b>PRIMARY KEY</b>          |
| <b>NOT NULL</b>    | Value cannot be NULL         | name TEXT <b>NOT NULL</b>              |
| <b>UNIQUE</b>      | All values must be unique    | email TEXT <b>UNIQUE</b>               |
| <b>DEFAULT</b>     | Sets a default value         | is_active BOOLEAN <b>DEFAULT TRUE</b>  |
| <b>CHECK</b>       | Ensures a condition is met   | age INTEGER <b>CHECK (age &gt;= 0)</b> |

## 2. Primary Key

- A Primary Key uniquely identifies each record in a table.
- It cannot have duplicate values.
- It cannot be NULL.
- Every table should have a primary key.
- A table can have only one primary key (but it can be a combination of columns).



**PRIMARY KEY**  
= Unique Identity

### Syntax:

```
column_name data_type PRIMARY KEY
```

Think of it like a unique ID for every row!

## 4. Complete Example – employees table



### Business Requirement:

- A company wants to store details of its employees including ID, name, salary, date of birth, joining date and active status.

### SQL Query:

```
CREATE TABLE employees (
    employee_id INTEGER PRIMARY KEY,
    employee_name TEXT NOT NULL,
    salary NUMERIC(10, 2),
    date_of_birth DATE,
    joining_date TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE
);
```

A well-designed table helps you store clean, consistent and useful data!

## 5. Inserting Data

- After creating the table, we can insert data using the **INSERT INTO** statement.

### Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

### Example:

```
INSERT INTO employees (employee_id, employee_name,
    salary, date_of_birth, joining_date, is_active)
VALUES
(1, 'Rahul', 50000.75, '1995-04-12', '2023-06-01 09:00:00', TRUE),
(2, 'Priya', 62000.00, '1993-11-23', '2022-01-15 10:30:00', TRUE),
(3, 'Aman', 45000.50, '1998-07-09', '2024-03-10 09:15:00', FALSE);
```

## 6. Result – employees table

| employee_id | employee_name | salary   | date_of_birth | joining_date        | is_active |
|-------------|---------------|----------|---------------|---------------------|-----------|
| 1           | Rahul         | 50000.75 | 1995-04-12    | 2023-06-01 09:00:00 | true      |
| 2           | Priya         | 62000.00 | 1993-11-23    | 2022-01-15 10:30:00 | true      |
| 3           | Aman          | 45000.50 | 1998-07-09    | 2024-03-10 09:15:00 | false     |



### Key Takeaway:

- Design tables based on business needs.
- Choose the right data types.
- Use constraints to ensure data quality.
- A good table design makes future analysis easier!



"Good database design is like a strong foundation – it supports everything you build next!"



PostgreSQL

# Topic 02 – Tables & Data Types

(Page 4 of 4)

Learn Today → Build Better Tomorrow !

"Good Databases  
Power Great Ideas!"

## 7. Common Errors (and How to Fix Them)

| Error                               | Example           | Why it's a Problem                     | Fix                            |
|-------------------------------------|-------------------|----------------------------------------|--------------------------------|
| Storing numbers as TEXT             | salary TEXT       | Cannot perform mathematical operations | Use NUMERIC or INTEGER         |
| Using INTEGER for decimal values    | price INTEGER     | Loses decimal part                     | Use NUMERIC(10,2)              |
| Not setting a Primary Key           | -- no primary key | Can have duplicate records             | Add PRIMARY KEY                |
| Allowing NULL in important fields   | name TEXT         | Missing important data                 | Use NOT NULL                   |
| Using very small VARCHAR            | name VARCHAR(5)   | May not store full data                | Use appropriate length or TEXT |
| Wrong data type for date/time       | joining_date TEXT | Cannot use date functions              | Use DATE or TIMESTAMP          |
| Duplicate values in a unique column | email TEXT        | Data inconsistency                     | Use UNIQUE constraint          |
| Forgetting DEFAULT                  | is_active BOOLEAN | Need to manually set value             | Use DEFAULT TRUE (for example) |

## 9. Real-World Use Cases

- HR System → Store employee details (ID, name, salary, joining date)
- E-commerce → Store customer, product, order information
- Banking → Store account details, transactions
- Healthcare → Store patient records, appointment details
- Education → Store student details, courses, enrollments
- Analytics → Store structured data for reporting and analysis

No matter the industry, everything starts with a well-designed table !

## 10. Think Like a Database Engineer

- ✓ Understand the business requirement.
- ✓ Choose the right data types.
- ✓ Use appropriate constraints.
- ✓ Design for data integrity and future scalability.
- ✓ Think about how the data will be queried and analyzed.
- ✓ Avoid common mistakes.
- ✓ Keep your database clean and well-structured.
- ✓ Good design now = fewer problems later !

Think  
Design  
Build  
Improve  
Repeat !

## 8. Interview Questions & Answers

- Q1:** What is a table?  
**A1:** A table is a collection of related data stored in rows and columns.
- Q2:** What is a column?  
**A2:** A column is a vertical set of values representing a specific attribute or property.
- Q3:** What is a row?  
**A3:** A row is a horizontal set of values representing one complete record.
- Q4:** What is a primary key?  
**A4:** A primary key uniquely identifies each record in a table. It cannot have duplicate or NULL values.
- Q5:** What is the difference between TEXT and VARCHAR(n)?  
**A5:** Both store text. VARCHAR(n) has a length limit, while TEXT can store unlimited length. In PostgreSQL, TEXT is often preferred for general text storage.
- Q6:** When should you use NUMERIC?  
**A6:** Use NUMERIC for decimal numbers (e.g., salary, price) where precision is important.
- Q7:** What is the difference between DATE and TIMESTAMP?  
**A7:** DATE stores only the date (YYYY-MM-DD), while TIMESTAMP stores both date and time (YYYY-MM-DD HH:MI:SS).
- Q8:** Can a table have multiple primary keys?  
**A8:** A table can have a composite primary key (multiple columns together), but only one primary key constraint.

"In interviews, explain the WHY, not just the WHAT !"

## 11. Quick Revision Cheat Sheet

| Data Type    | Use For                     | Example               |
|--------------|-----------------------------|-----------------------|
| INTEGER      | Whole numbers               | 25                    |
| BIGINT       | Very large numbers          | 9000000000            |
| NUMERIC(p,s) | Decimal numbers (precision) | 1250.75               |
| TEXT         | General text                | 'Hello World'         |
| VARCHAR(n)   | Text with length limit      | VARCHAR(100)          |
| BOOLEAN      | True / False                | TRUE                  |
| DATE         | Date only                   | '2026-08-26'          |
| TIMESTAMP    | Date and time               | '2026-08-26 10:30:00' |

### ★ Final Takeaway

- Tables store the data.
- Data types define the kind of data.
- Constraints keep the data clean and reliable.
- Good table design makes your database powerful, efficient, and future-ready !

Design  
Good Tables  
Build  
Great Solutions !

"A well-structured table today can handle tomorrow's biggest data challenges !"

Keep Learning ! ❤️